PROJECT BRIEFING · READ BEFORE THE REVIEW

CHIMERA, explained simply

What the project actually is, why it exists, and what we found — written for someone seeing it for the first time. No control theory required.

Operating Systems Laboratory · BCSE303P · about 10 minutes to read

The one-paragraph version

When a website gets busy, something has to give it more computing power. Kubernetes — the system most companies use to run their software — has two separate ways to do that, and they don't coordinate with each other. CHIMERA replaces both with a single controller that drives them together, and that knows one of the two is much slower than the other. That's the whole idea. Everything else is detail.

The problem, using a kitchen

Imagine a restaurant kitchen that suddenly gets slammed with orders. You have two ways to cope.

THE ANALOGY

Option one: hire more cooks. Unlimited upside — you can always hire another. But a new cook takes twenty minutes to arrive, change, and actually start cooking. By the time they're useful, the rush might be over.

Option two: give your current cooks a bigger stove. Instant — you flip a switch and they can cook faster right now. But there's a ceiling. The kitchen only has so much space, and a bigger stove can't replace a whole extra person.

Neither works alone. You want the stove *immediately* to survive the next twenty minutes, and the cooks *arriving* to handle the rest.

In Kubernetes those two options have names. Hiring cooks is **horizontal scaling** — adding more copies of your program, called pods. Each new pod takes about 20 seconds to boot. Upgrading the stove is **vertical scaling** — giving existing pods more CPU. That takes about 0.1 seconds.

Horizontal — add pods

20 s

Unlimited ceiling, but slow. You're committing to capacity that won't arrive for twenty seconds.

Vertical — resize CPU

0.1 s

Effectively instant, but capped by how much CPU one machine has spare.

One is two hundred times slower than the other. That gap is the entire problem.

Why what exists today doesn't work

Kubernetes ships both controllers, and they run as completely independent programs. Neither knows the other exists. Back in the kitchen, that's two managers:

WHAT GOES WRONG

Manager A sees a queue and hires five cooks. Manager B, looking at the same queue at the same moment, cranks every stove to maximum.

Twenty minutes later the five cooks arrive — and the stoves are *still* maxed out. Now the kitchen is wildly over-resourced, so both managers panic and cut back at the same time. Now it's under-resourced. So they both scale up again.

The kitchen never settles. Engineers call this **oscillation**, or "control loops fighting", and it's why Kubernetes' own documentation warns you not to run both controllers on the same resource.

So today you effectively pick one and give up the other. That's the gap CHIMERA is trying to fill.

What CHIMERA does differently

Two ideas, and they're both simple once you strip the maths away.

One manager instead of two

Rather than two controllers each solving half the problem, CHIMERA computes both decisions at once, from one calculation. It can deliberately say "add 0.5 CPU *and* two pods" as a single coordinated move, because it's weighing both against each other. Two independent controllers structurally cannot do that.

Remembering what's already on its way

This is the actual novel bit, and it's the thing worth being able to explain.

If you order five cooks and then look at the kitchen ten seconds later, it still looks understaffed — because they haven't arrived yet. A naive controller panics and hires five more. Twenty seconds later, ten cooks walk in.

CHIMERA keeps a short list of pods it has already requested but that haven't booted yet, and factors them in. So when it looks at the kitchen it effectively thinks "yes, we're short *right now*, but five are already walking through the door — don't order more." In the write-up this is called **delay-aware state augmentation**, or DAPCA. In plain terms: it remembers what it already ordered.

> IF YOU REMEMBER ONE THING
>
> This is the component our own experiments show matters most. Switch it off and the failure rate jumps by almost 11 percentage points — by far the biggest single effect we measured. If you're asked "what's actually new here," this is the answer.

How it's put together

Three stages, running in a loop every 15 seconds.

1. **Predict.** Look at recent traffic and forecast the next stretch — importantly, as a *range* rather than a single guess, so the system knows how confident it is.
2. **Decide.** Work out the cheapest combined move — how much CPU, how many pods — that gets back on target. Then run it past a safety check that rejects any move which would make things less stable.
3. **Act.** Resize the CPU, request the pods, and write down what was ordered so the next cycle remembers.

All of this is tested in a simulator — a program that pretends to be a Kubernetes cluster, complete with realistic 20-second pod boots and requests queuing up. That lets us run thousands of experiments in minutes instead of renting real servers. There's also a real Kubernetes component written, but it hasn't been run on a live cluster yet.

What we actually found

This part matters, because it's the part people get wrong when they present it. We ran 720 simulations comparing CHIMERA against six other approaches on real traffic data from Microsoft Azure.

MEASURE	CHIMERA	KOSMOS	WHAT IT MEANS
Worst-case slowness	**3.6 s**	21.6 s	Our unluckiest users wait 6× less
Scaling changes made	**13**	261	20× less churn on the cluster
Requests missing target	9.7 %	13.6 %	Better, but see below
Resources used	147	97	We spend 52 % more to get it

The honest reading: CHIMERA does not beat everything at everything. One competitor gets a slightly better failure rate than we do — by spending 85% more computing resource to get there. We're better than another competitor on failures, but we spend more than it does.

Where CHIMERA wins clearly, and where nobody can argue, is the two rows at the top. A 6× better worst case, and 20× fewer disruptive changes to the running system. That second one matters more than it sounds: every scaling change is a small disturbance to live software, and a system making 261 of them per run is thrashing.

WHY WE PRESENT IT THIS WAY

Speed and cost trade off against each other. Any system can look great on failure rate by just buying more servers. So quoting a failure rate without also quoting the cost is meaningless — and an examiner who knows this will ask. We put both numbers on the slide so the question is already answered.

What's done and what isn't

Ten of twelve components are finished and tested. Six automated tests pass. The results database is real and reproducible.

Two things are genuinely not done, and we say so on the slides rather than hiding them:

- **It has never run on a real cluster.** The Kubernetes pieces are written and packaged, but the part that reads live traffic still returns fake numbers. Everything measured so far is simulation.
- **Two components don't earn their keep.** Our own ablation tests — switching each piece off to see what happens — show one component makes no measurable difference, and another slightly hurts. Both are flagged for rework.

We also found and fixed four real bugs during the project, including one where new pods were becoming ready in 0.1 seconds instead of the realistic 20, which had been quietly flattering the competing approaches. There's a slide on this. It's a strength, not an admission — finding your own bugs is what separates a project that ran experiments from one that verified them.

Questions you might get, and what to say

"How is this an Operating Systems project?"

An OS decides which programs get CPU and memory and when. This is exactly that problem, one level up — deciding how much CPU each piece of software gets across a cluster of machines, under the constraint that granting it takes time. Same problem, bigger box.

"Why does another approach beat you on failure rate?"

Because it spends 85% more computing resource. Compare them at equal cost and the picture changes. That's why our results slide shows cost and quality side by side rather than a single number.

"What's genuinely new here?"

Driving both scaling methods from one calculation, and building the 20-second delay into that calculation so the controller accounts for capacity it has already ordered. Other systems do one or the other; we're not aware of one doing both together.

"Does it work in production?"

Unknown — and we say so. It works in simulation across 720 runs. Deploying on a real cluster is the next milestone.

"Why not use machine learning for the whole thing?"

We do use ML for the traffic forecast. But the decision itself uses control theory, which gives mathematical guarantees about stability that a neural network can't. For something that controls live infrastructure, predictable beats clever.

Words you'll hear on the slides

Pod

One running copy of the software. Adding pods is horizontal scaling.

SLO

Service Level Objective — the promise, here "requests finish in under half a second." A violation is a request that missed it.

LQR

A standard control-engineering method for working out the cheapest correction that gets a system back on target. Decades old, widely used in aerospace and robotics.

Lyapunov filter

A safety check. Before acting, it asks "would this move take us further from stable?" and blocks it if so.

Conformal prediction

A forecast that comes with an honest error bar — not just "expect 100 requests" but "expect 80 to 130, and we're right 90% of the time."

Ablation

Switching one component off to measure how much it was contributing. How we know which parts matter.

KOSMOS

A published 2021 system that also does both kinds of scaling. Our main comparison; we reimplemented it to compare fairly.

p99

The worst-case experience. If p99 is 3.6 seconds, 99% of requests were faster than that and the unluckiest 1% were slower.

Every number here comes from the project's verified results file — 720 simulation runs across 27 configurations. If something in this doc contradicts a slide, trust the slide and flag it.